Gemma 4 Developer Agent Competition

Methodology TBD

Wenjia Jiang (University of Alberta)
Di Liao (Hangzhou Dianzi University)
Sholpan Amankeldi (Westlake University)
Junzhe Jiao (Tongji University)
Xuanlin Guo (Fudan University)



Our first meeting: understand, explore, try



UNDERSTAND

How does an agent fix code?

Follow a real repair, one tool call at a time.



EXPLORE

What does Kaggle score?

Meet the tasks, resources, and our first result.



TRY

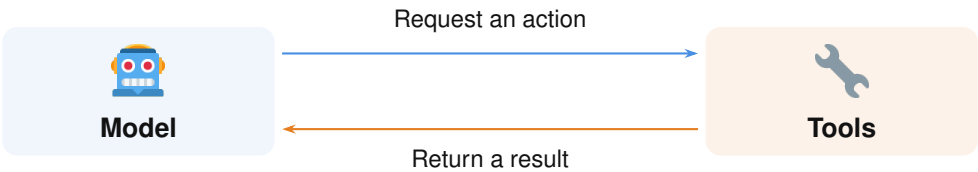
What could make it better?

Choose two ideas we can test together.

01 How a coding agent works

A coding agent acts, observes, and tries again

A large language model (LLM) predicts text and structured tool requests.



THE MODEL DECIDES

Which file looks relevant? What should change?

TOOLS DO THE WORK

Search, read files, edit code, and run tests.

During a repair, the model reads new results; its learned parameters stay fixed.

A tool call, shown as a conversation

From our completed run: the agent is looking for the URL highlighter.

Agent asks run_command

```
find rich -name "highlighter.py"
```

Tool replies

```
rich/highlighter.py
```

Agent continues read_file

Read this file and inspect the matching pattern.

One small repair makes the loop concrete

Public task `rich_3454`; the URL below illustrates the bug.

Before the fix

The match stops before @.

`https://example.com/@user`

After the fix

The whole URL is highlighted.

`https://example.com/@user`

EDIT, CHECK, FINISH

Allow @ in the matching pattern and add a regression test.

`pytest tests/test_highlighter.py` ⟶ **Tests pass**

The agent finishes and exports a patch: its changes as a Git diff.

02 Meet the competition

The task: turn an issue into a working patch

Kaggle gives the agent an issue and a repository before the fix.



Read
Issue and hints
Pre-fix repository

Repair
Edit and test
Repeat as needed

Export
Final code changes
A Git diff patch

Verify
Clean test run
PASS / FAIL

Score = fraction of tasks resolved by the final patches.

Our entry tells Gemma how to work

ONE AGENT ZIP

Instructions and settings tell the agent how to search, edit, and test.

An optional learned adapter adds training updates.

Training is optional.
Our scored starting agent uses instructions and settings.

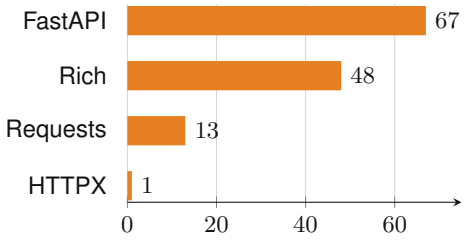


Required model
Official quantized Gemma 4 31B.

Upload ZIP ⟶ **Run agent on each issue** ⟶ **Generate and grade patches**

Reuse the same agent configuration for every task; put `agent.yaml` at the ZIP root.

Public data: 129 tasks across four repositories



WHAT IS INCLUDED?

- Issue descriptions and source snapshots
- Reference fixes and verification tests
- Search data and runtime packages
- Example agent and setup guide

Most examples come from FastAPI and Rich. Try other repositories too.

How do we know an improvement is real?

CHECK THE TEST FIRST

Original code reproduces the bug.

The reference repair passes the same check.

Agent input: the issue, original code, and tools.

KEEP THE FINAL CHECK FAIR

Example split

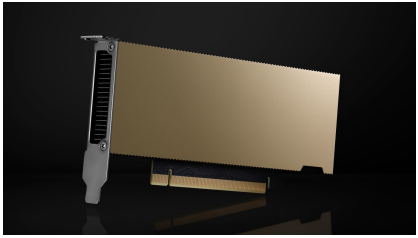
Related code snapshots ⟶ training set

A different repository ⟶ evaluation set

Use held-out answers only for grading.

> **A near-duplicate repair can inflate the score. Fresh tasks test whether the idea transfers.**

The competition gives us four L4 GPUs



4 × L4 24 GB each

32,768 tokens	Text per model call
12 hours	Setup and patch generation
Under 3 GiB	Unpacked submission size

Tokens are pieces of text. The context holds the issue, conversation, tool results, and response.

About 120 hidden tasks means roughly six minutes per repair, before overhead.

Your first Kaggle session

OPEN THE OFFICIAL STARTER

1. Copy the Getting Started notebook.
2. Attach data, packages, and the required model.
3. Select **L4 × 4** and **Internet off**.
4. Try a few tasks; read the patches and tests.

Check GPU time first

Check your remaining GPU quota before starting. L4 × 4 uses twice the T4 × 2 / P100 reference rate.

The scored run has its own 12-hour budget.

A first exercise: follow one repair from start to finish

In the official starter, complete setup and start the model server first.

CHOOSE ONE TASK IN SECTION 5

Replace the `SAMPLE_TASKS` selection:

One public example

```
SAMPLE_TASKS = [t for t in tasks
if t.instance_id == "rich_3454"]
```

RUN AND INSPECT

Run the inference and verification cell.

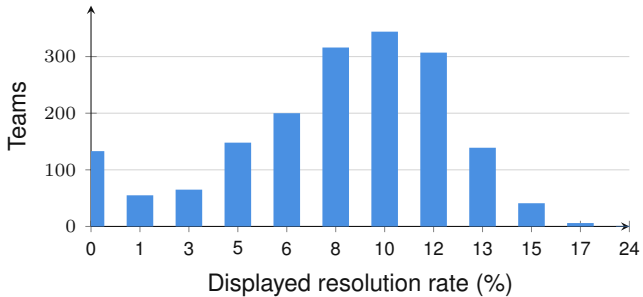
Read the tool log, final patch, and test result.

Change one instruction; repeat the same task to learn the workflow.

Finish with a patch and verification result.
Use shared held-out tasks for later comparisons.

Leaderboard snapshot: best 24%, median 8%

October 5, 2026, 03:50 UTC / 1,755 scored teams / public leaderboard



HOW TO READ IT

Many teams are tied. A few solved tasks can move the rank.

Use local tests to understand why an idea helps.

Our first milestone: a scored agent and a repair

OFFICIAL PUBLIC SCORE

0.12

Our baseline submission completed successfully.

12% public resolution rate.

A COMPLETED KAGGLE REPAIR

Run on **4 NVIDIA L4 GPUs**.

Public task `rich_3454`:

84 tests passed

Agent phase: **74.10 seconds**.

One task; timing excludes model startup.

Next: evaluate more tasks and try our first small training experiment.

A reading map for the team

START WITH THREE THINGS

Competition rules ⟶ Official starter ⟶ Our first result

112 notebooks

Find ideas and code examples.

116 discussions

Read organizer answers and lessons.

4 model cards

Understand available model resources.

Open `resource/README.md` for the October 5 archive and review notes.

03 Ideas to explore

SFT, LoRA, and RL in plain language

SFT: LEARN BY EXAMPLE

Show good repairs

Supervised fine-tuning teaches useful tool calls and edits from checked examples.

LoRA: A SMALL UPDATE

Train extra weights

A compact adapter changes the model's behavior while the base stays fixed.

RL: LEARN BY FEEDBACK

Reward useful actions

Reinforcement learning improves choices using feedback, such as test results.

How they fit together

SFT or RL defines how the model learns. LoRA defines which weights we train.

Three ideas to compare with our starting agent

Change one thing at a time; use the same model, tasks, and time budget.

Our starting agent

Public score: 0.12

Better search

Can clearer search instructions save time?

Code relationships

Can links between functions improve repairs?

Learn from examples

Can a small adapter help on fresh tasks?

Compare repair success, time spent, and what went wrong.

Turn checked repairs into learning material

A trace records the agent's actions, tool results, and code changes.

1 → 2 → 3 → 4

Choose tasks
Keep related code
in the same split

Check repairs
Read the trace
Run the tests

Train an adapter
Start with SFT
on good examples

Try fresh tasks
Measure repairs
within a time limit

SAVE FRESH TASKS FOR LATER

Keep related code together when making training and evaluation sets.

LEARN FROM CHECKED OUTCOMES

Confirm that the patch works and the test environment behaves correctly.

04 Planning our first training experiment

A training workspace your team can open in Kaggle

PREPARE THE NOTEBOOK

1. Keep the working starter for evaluation.
2. Create a training notebook for this competition.
3. Choose L4 $\times$ 4 and Internet off.

ADD INPUT BEFORE STARTING

Bring three ingredients

Checked training examples
Compatible Gemma weights
Offline training packages

First check model and package compatibility, then complete one small training update.

What does one training example look like?

Teach the next useful action from a checked repair trace.

CONTEXT

Issue + earlier tool output

URL highlighting breaks at @.

File search returned:
`rich/highlighter.py`

NEXT ACTION

The training target

Call `read_file`

with `filepath` set to
`rich/highlighter.py`

SFT uses tool results as context and trains on checked assistant actions.

With LoRA, training updates a small adapter while the base model stays fixed.

Planned: our first small training run

TRL handles training; PEFT trains and exports the LoRA adapter.

One update

Small batch
Check memory

→

A tiny pilot

10–20 steps
Watch the loss

→

Save + reload

Export weights
Fresh session

→

Compare

Fresh tasks
Same time limit

Loss measures fit to training examples. Test repair success on fresh tasks.

SAVE

Write to `/kaggle/working/`.
Save Version with outputs before stopping.

RE-EVALUATE

Attach the adapter to the official starter.
Compare the same held-out tasks.

05 Working together

Find your way around the project in 15 minutes

Start here

`README.md`

Project map and current progress.

Explore useful examples

`resource/README.md`

Official sources and public notebooks.

Understand the competition

`docs/overview.md`

`docs/data.md`

Rules, tasks, and what the files mean.

Try a change

`configs/baseline/`

Agent instructions and settings.

Where should I put my work?

SHARE THROUGH GIT

`src/`	Our code
`configs/`	Agent and training settings
`scripts/`	Commands to prepare and run
`docs/`	Guides and findings
`resource/`	Reading and source indexes
`slides/`	This presentation

KEEP LARGE FILES SEPARATE

`data/`	Original and prepared data
`model/`	Base weights and adapters
`runs/`	Logs, results, and checkpoints
`cache/`	Temporary files

Document where teammates can obtain each large input.

Share code and short result summaries. Keep API tokens in personal settings.

Make each experiment easy for a teammate to follow

We plan to use GitHub for shared code and review; local Git is initialized.

1 → 2 → 3 → 4

Pick a question
Open an issue
Define success

Try an idea
Use a branch
Save the settings

Show evidence
Open a pull request
Explain the results

Merge
Review together
Share the change

A useful experiment note

What changed? What improved? What did we learn?

Our next steps, with the deadlines in view

TEAM PLAN 🚀

Build confidence
Evaluate the starting agent on more tasks.

Explore improvements
Try one search or strategy change.

Learn from examples
Train, save, and evaluate a small adapter.

Prepare the final entry
Check repair quality and total runtime.

2026 DATES / 23:59 UTC

Nov 12 Optional paper track entry

Nov 25 Entry and team merger

Dec 02 Final prediction deadline
One submission per day.

Two final selections.
Up to five teammates.

Which two ideas should we try together?

FIND THE RIGHT CODE

Better file search

Can we reach useful code with fewer tool calls?

FOLLOW CONNECTIONS

Code relationships

Can links between functions lead to better repairs?

LEARN FROM REPAIRS

A small training pilot

What examples would help the agent most?

Choose two ideas, a shared task set, and what success would look like.

Reference links and project reading list

OFFICIAL LINKS

Competition Overview / Evaluation
Data and description
Rules and team requirements
Upgraded Accelerators: L4 × 4
Official Getting Started notebook
Official evaluation wheelhouse
Live leaderboard

PROJECT AND TRAINING GUIDES

`README.md`
`docs/overview.md`
`docs/data.md`
`docs/participation.md`
`docs/competition-research.md`
`resource/README.md`
TRL: supervised fine-tuning
PEFT: LoRA and quantization

Images: Google Gemma 4, NVIDIA L4. Emoji: Twemoji, CC BY 4.0.